Florence-2 NSSF OCR

Fine-Tuning Documentation

Complete Guide · Training · Dataset Format · Re-Fine-Tuning

Base Model	Dataset	Platform	Task Token
microsoft/Florence-2-large-ft	8,125 NSSF Documents	RunPod / Single GPU	<Extract_NSSF_Data>

Model to be finetuned	Dataset	Platform	Task Token
florence2-nssf-ocr-final	Your dataset Documents	RunPod / Single GPU	<Extract_NSSF_Data>

1. Overview

This notebook fine-tunes Microsoft Florence-2-large-ft on 8,125 NSSF (National Social Security Fund) document images. The model learns to extract structured JSON data from mixed print and handwritten forms, tables, and scanned documents using a custom task token.

What This Notebook Does

- Downloads and extracts the 8,125-image NSSF training dataset from the local pc storage
- Loads Florence-2-large-ft and registers a custom task token **<Extract_NSSF_Data>**
- Applies LoRA (Low-Rank Adaptation) with r=16, alpha=32 for efficient fine-tuning
- Trains for 7 epochs with gradient checkpointing, cosine LR schedule, and W&B logging
- Evaluates using Exact Match, CER (Character Error Rate), and Fuzzy Match metrics
- Saves the final merged model ready for production inference

8 Critical Bug Fixes Applied

> **i Info:** This notebook corrects 8 bugs present in earlier versions. Understanding these fixes helps you avoid common pitfalls when extending the training.

#	Bug Description	Fix Applied
1	Full 365-token Qwen prompt fed to Florence (context overflow)	Stripped → replaced with <Extract_NSSF_Data>
2	processor() call missing max_length	Added max_length=512, truncation=True to label tokenizer only

#	Bug Description	Fix Applied
3	LoRA r=8, alpha=8 → scale=1.0 (underpowered for handwriting)	Upgraded to r=16, alpha=32 (scale=2.0)
4	max_steps=1000 = only 0.98 epochs on 8,125 samples	Changed to num_train_epochs=5 (then 7 in final)
5	gradient_checkpointing=True missing from TrainingArguments	Added to TrainingArguments
6	model.config.use_cache=False missing (crashes with grad-checkpointing)	Added before LoRA wrapping
7	model.resize_token_embeddings() missing after adding task token	Added after tokenizer update
8	Image path not prefixed with BASE_DIR (fails on RunPod)	All paths prefixed with BASE_DIR

2. Prerequisites & Environment Setup

Hardware Requirements

Spec	Requirement
GPU	NVIDIA GPU with ≥ 16 GB VRAM (RTX 4090, A100, RTX PRO 4500, or T4)
CUDA	≥ 12.8 for Blackwell GPUs (sm_120); 11.x for older GPUs
RAM	≥ 32 GB system RAM recommended
Storage	≥ 50 GB free disk space for dataset + model checkpoints
Platform	RunPod recommended; any Ubuntu 22.04+ single-GPU instance works

GPU-Specific Batch Sizes

GPU	VRAM	BATCH_SIZE	GRAD_ACCUM	Effective Batch
T4	16 GB	1	8	8
RTX 4090 / A10	24 GB	2	4	8
RTX PRO 4500	34 GB	4	2	8
A100 / A40	40+ GB	4	2	8

Software Dependencies

All dependencies are installed in Cell 0. The key libraries are:

- **PyTorch ≥ 2.7.0** — Required for Blackwell GPU (sm_120) support, installed from CUDA 12.8 wheels

- **transformers==4.49.0** — Pinned version for Florence-2 compatibility
- **peft ≥ 0.10.0** — LoRA implementation via HuggingFace PEFT
- **accelerate ≥ 0.27.0** — multi-GPU and mixed-precision training
- **bitsandbytes ≥ 0.43.0** — 4/8-bit quantisation support
- **wandb** — Experiment tracking and loss curve visualization
- **rapidfuzz** — Fast fuzzy matching for evaluation metrics
- **py7zr** — 7-zip archive extraction for dataset

> ⚠ **Warning:** After running Cell 0 (dependency installation), you MUST restart the kernel before continuing. The old PyTorch version remains cached in memory until restart.

Weights & Biases Setup

The notebook integrates with Weights & Biases (W&B) for experiment tracking. To enable:

1. Create a free account at https://wandb.ai
2. Go to Settings → API Keys and copy your key
3. In Cell 2 (Configuration), set: WANDB_API_KEY = 'your_key_here'
4. Optionally set WANDB_ENTITY to your W&B username or team name

> **i Info:** If you prefer not to use W&B, set USE_WANDB = False in Cell 2. Training will continue normally but without the live dashboard.

3. Configuration Reference

All training parameters are centralized in Cell 2. Adjust these before running the notebook.

Path Configuration

Variable	Description
BASE_DIR	Auto-set to /workspace on RunPod, or ~ on other machines
DATA_JSON	Path to ultimate_training_dataset.json (your training data)
IMAGE_BASE_DIR	Root directory where all image files are stored
OUTPUT_DIR	Checkpoints saved here during training
FINAL_MODEL	Final merged model saved here after training
FLORENCE_BASE	Base model: microsoft/Florence-2-large-ft

Training Hyperparameters

Parameter	Default Value	Description
BATCH_SIZE	4	Per-GPU batch size (reduce if OOM)
GRAD_ACCUM	2	Gradient accumulation steps
NUM_EPOCHS	7	Total training epochs
LR	1e-4	Peak learning rate (cosine schedule)
WARMUP_STEPS	100	Steps before LR reaches peak
SAVE_STEPS	500	Checkpoint save interval
LOG_STEPS	10	W&B logging interval
MAX_INPUT_LEN	1024	Max tokens for image+prompt input
MAX_LABEL_LEN	640	Max tokens for JSON label output
VAL_SPLIT	0.05	5% of data held out for validation

LoRA Configuration

Parameter	Value / Notes
LORA_R	16 — Rank of adaptation matrices (higher = more capacity)
LORA_ALPHA	32 — LoRA scaling factor (alpha/r = 2.0 scale)
LORA_DROPOUT	0.05 — Dropout probability on LoRA layers
LORA_TARGETS	q_proj, k_proj, v_proj, out_proj, fc1, fc2, linear, Conv2d
Task Type	CAUSAL_LM — matches Florence-2's generation mode

> **Tip:** The LoRA scale (alpha/r = 2.0) is intentionally set higher than the default 1.0. This is important for handwritten text recognition, which requires stronger adaptation than typical print-only OCR.

4. Dataset Format

The training dataset uses a JSON format compatible with Qwen/LLaVA-style multi-modal conversation datasets. This same file can be used to fine-tune other vision-language models (LLaVA, Qwen2-VL) without modification.

File Structure Overview

- **File name:** ultimate_training_dataset.json
- **Format:** JSON array of entries
- **Current size:** 8,125 entries (images + structured JSON labels)
- **Image types:** Mixed print, handwriting, tables, scanned forms

Single Entry Schema

Each entry in the JSON array has this structure:

```
{
 "images": ["nssf_dataset/image_001.jpg"],
 "messages": [
   {
    "role": "user",
    "content": "You are an expert data extraction assistant..."
   },
   {
    "role": "assistant",
    "content": "{
      \"Document_Type\": \"NSSF_Form_6\",
      \"Institution\": \"ABC Company Ltd\",
      \"Employer_Number\": \"532061\",
      \"Employee_Name\": \"John Doe\",
      \"National_ID\": \"12345678\",
      \"Period\": \"January 2024\",
      \"Table\": [...]
    }"
   }
 ]
}
```

Field-by-Field Breakdown

Field	Type	Required	Description
images	Array[string]	Yes	Relative path(s) to image file(s). Path is joined with IMAGE_BASE_DIR at runtime.
messages	Array[object]	Yes	Conversation array with exactly 2 entries: user prompt and assistant response.
messages[0].role	string	Yes	Must be 'user'
messages[0].content	string	Yes	The instruction prompt. Any text containing 'You are an expert data extraction assistant' is auto-detected and replaced with <Extract_NSSF_Data> during training.
messages[1].role	string	Yes	Must be 'assistant'

Field	Type	Required	Description
messages[1].content	string	Yes	Valid JSON string. This is the ground truth the model learns to produce.

Supported Image Formats

- **.jpg / .jpeg** — Most common, recommended for photographed documents
- **.png** — Best for scanned documents and forms
- **.tif / .tiff** — Supported for high-resolution archival scans

Ground Truth JSON Labels

The assistant content (labels) must be a valid JSON string containing NSSF field extractions. Typical fields include:

```
{
 "Document_Type": "NSSF_Form_6",
 "Institution": "ABC Company Ltd",
 "Employer_Number": "532061",
 "Employee_Name": "John Doe",
 "National_ID": "12345678",
 "Postal_Address": "P.O. Box 1234-00100",
 "Period": "January 2024",
 "Total_Contributions": "15000",
 "Table": [
   {
    "Name": "Jane Smith",
    "ID": "87654321",
    "Employee_Contribution": "200",
    "Employer_Contribution": "200"
   }
 ]
}
```

Image Path Structure

The images array contains paths relative to IMAGE_BASE_DIR. The dataset uses a nested folder structure:

```
/workspace/                  ← IMAGE_BASE_DIR (BASE_DIR)
 nssf_dataset/
  nssf_dataset/              ← double-nested (7z extraction artefact)
    image_001.jpg
    image_002.jpg
    ...
 ultimate_training_dataset.json
```

> **i Info:** The dataset loader automatically prepends IMAGE_BASE_DIR to each image path at runtime. The JSON file stores only relative paths like 'nssf_dataset/image_001.jpg'.

Dataset Validation

Cell 3 runs automatic validation before training starts:

- **Image existence check:** Verifies every image path resolves to a real file
- **JSON parse check:** Ensures every label is valid parseable JSON
- **Missing file report:** Lists up to 5 missing paths so you can investigate

> ⚠ **Warning:** Always run Cell 3 validation after adding new data. Training on missing-image entries silently corrupts the loss curve.

5. Step-by-Step Training Walkthrough

Cell 0 — Install Dependencies

Uninstalls any existing PyTorch, then installs the CUDA 12.8 wheel (required for Blackwell RTX PRO 4500 / sm_120). Then installs transformers, peft, accelerate, wandb, and all supporting libraries.

> ⚠ **Warning:** Restart the kernel after this cell completes. The print statement at the end reminds you.

Cell 0b — Download & Extract Data

Downloads two archives from Google Drive using gdown:

- **nssf_dataset.7z** — The 8,125 training images (~several GB)
- **custom_nssf_model.7z** — A previously trained 87-sample Florence model (optional reference)

The smart_extract() function detects archive format by magic bytes (not extension) and extracts safely into the destination folder.

Cell 1 — Imports & Device Detection

Imports all required libraries and detects the GPU. Prints VRAM, compute capability (sm_XX), and recommends batch sizes based on detected VRAM.

Cell 2 — Configuration

All tunable parameters in one place. See Section 3 (Configuration Reference) for full details. Edit WANDB_API_KEY, BATCH_SIZE, NUM_EPOCHS, and paths before running.

Cell 3 — Dataset Validation

Loads ultimate_training_dataset.json and validates every entry. Reports missing images and malformed JSON labels. Safe to re-run at any time.

Cell 4 — NSSFDocumentDataset

Defines the PyTorch Dataset class. Key behaviors:

- Detects the long 365-token Qwen instruction and replaces it with the compact **<Extract_NSSF_Data>** task token
- Loads images with PIL and converts to RGB
- Applies optional data augmentation (ColorJitter, RandomRotation ±3°, GaussianBlur) during training
- Augmentation is disabled for the validation split

Cell 5 — Data Collator

The collate_fn function batches (prompt, label, image) tuples into Florence-2 model inputs. Critical implementation notes:

- **max_length is NOT passed to processor():** Florence-2's processor subtracts 577 image tokens from max_length internally — passing 512 would produce -65, causing an overflow error in the Rust tokenizer
- **max_length IS passed to the label tokenizer:** Labels contain no image tokens, so truncation to MAX_LABEL_LEN=640 is safe
- **Padding tokens replaced with -100:** Loss computation ignores padding positions

Cell 6 — Load Florence-2 + Apply LoRA

Loads the processor and model, then applies LoRA. Critical steps in order:

5. Load processor from microsoft/Florence-2-large-ft
6. Register **<Extract_NSSF_Data>** as an additional special token
7. Load model with fp32 base weights
8. Call resize_token_embeddings() to accommodate the new token
9. Set model.config.use_cache = False (required for gradient checkpointing)
10. Wrap model with LoRA via get_peft_model()
11. Call enable_input_require_grads() for PEFT + gradient checkpointing compatibility

Cell 7 — Split & Build Datasets

Shuffles the full dataset with seed=42, then splits 95% train / 5% validation. Runs a single-batch sanity check to confirm the data pipeline works before starting the full training run.

Cell 8 — TrainingArguments

Configures HuggingFace Trainer settings. Key settings:

- **fp16=True** — Mixed precision training. Switch to bf16=True on Ampere/Ada GPUs for better numerical stability
- **gradient_checkpointing=True** — Trades compute for memory; required on 16-24 GB GPUs
- **lr_scheduler_type='cosine'** — Cosine annealing with warmup
- **load_best_model_at_end=True** — Restores the checkpoint with the lowest eval_loss after training

- **remove_unused_columns=False** — Critical: prevents Trainer from dropping pixel_values

Cell 9 — Train

Initializes W&B, creates the Trainer, and starts training. Set RESUME=True if RunPod was restarted mid-training to continue from the last checkpoint.

Cell 10 — Save Final Model

Merges LoRA adapter weights into the base Florence-2 model with merge_and_unload(), then saves the complete model. Also generates a local loss curve PNG and links to the W&B interactive dashboard.

> **i Info:** Saving via merge_and_unload() instead of save_model() is important. Saving only the adapter causes a vocab size mismatch (51290 vs 51289) when reloading because the base model is loaded fresh before the adapter is applied.

6. Re-Fine-Tuning the Already Fine-Tuned Model

You can continue training the saved florence2-nssf-ocr-final model on new data, either to improve performance on existing document types or to add support for new document classes. This section covers the exact steps.

6.1 When to Re-Fine-Tune

- **More data available:** You've collected additional labeled NSSF documents and want to incorporate them
- **New document types:** New form variants (e.g. Form 7, EFT Reconciliation) that the model hasn't seen
- **Accuracy improvements:** CER is above 0.15 on specific document categories
- **Domain shift:** The scanned documents now come from a different scanner/quality level

6.2 Preparing New Training Data

Step 1 — Collect and Label Images

For each new document image, create a JSON label with all extractable fields. Labels must be valid JSON strings:

```
# Example label for a new Form 7 document
{
  "Document_Type": "NSSF_Form_7",
  "Institution": "New Company Ltd",
  "Employer_Number": "789012",
  "Period": "March 2024",
  "Total_Employees": "45",
  "Total_Contributions": "90000"
}
```

Step 2 — Add Entries to the Dataset JSON

Append your new entries to ultimate_training_dataset.json (or create a new file). Each entry must follow the schema from Section 4:

```
[
  // ... existing 8,125 entries ...
  {
    "images": ["nssf_dataset/new_form_7_001.jpg"],
    "messages": [
      {
        "role": "user",
        "content": "You are an expert data extraction assistant..."
      },
      {
        "role": "assistant",
        "content": "{\"Document_Type\": \"NSSF_Form_7\", ...}"
      }
    ]
  },
  // ... more new entries ...
]
```

> 💡 **Tip:** The user content can be any text containing 'You are an expert data extraction assistant' — the NSSFDocumentDataset class automatically strips it and replaces it with the task token. You can also write a simple script to generate these entries in bulk.

Step 3 — Place Image Files

Copy your new image files into the IMAGE_BASE_DIR so that the paths in the JSON resolve correctly:

```
# On RunPod:
cp /path/to/new_images/*.jpg /workspace/nssf_dataset/

# Then verify the path resolves:
import os
assert os.path.exists('/workspace/nssf_dataset/new_form_7_001.jpg')
```

6.3 Modifying the Notebook to Load the Fine-Tuned Model

To continue training from the saved fine-tuned model instead of the original Florence-2 base, change Cell 2 configuration and Cell 6 loading:

Change 1 — Update FLORENCE_BASE in Cell 2

```
# ── BEFORE (original training) ──────────────────────────────────────────
```

```
FLORENCE_BASE = 'microsoft/Florence-2-large-ft'

# ── AFTER (re-fine-tuning from saved model) ──────────────────────────────
FLORENCE_BASE = '/workspace/florence2-nssf-ocr-final'   # path to saved model

# Or if you've uploaded it to HuggingFace Hub:
FLORENCE_BASE = 'your-username/florence2-nssf-ocr-final'
```

Change 2 — Update DATA_JSON to Point to New/Extended Dataset

```
# Option A: Extend the original dataset (recommended — prevents forgetting)
DATA_JSON = os.path.join(BASE_DIR, 'ultimate_training_dataset_v2.json')

# Option B: Train only on new data (risk of catastrophic forgetting)
DATA_JSON = os.path.join(BASE_DIR, 'new_documents_only.json')
```

> ⚠ **Warning:** Training only on new data (Option B) risks 'catastrophic forgetting' — the model may lose accuracy on original document types. Always prefer mixing old and new data (Option A) unless you are certain the new data distribution covers all document types.

Change 3 — Reduce Learning Rate for Re-Fine-Tuning

When continuing from a trained model, use a lower learning rate to avoid overwriting learned representations:

```
# ── BEFORE (initial fine-tune) ──────────────────────────────────────────
LR = 1e-4

# ── AFTER (re-fine-tuning) ──────────────────────────────────────────────
LR = 3e-5    # 3x lower to preserve existing knowledge
WARMUP_STEPS = 50   # fewer warmup steps on smaller delta
NUM_EPOCHS = 3      # fewer epochs needed if delta is small
```

Change 4 — Update Output Directories

```
OUTPUT_DIR  = os.path.join(BASE_DIR, 'florence2-nssf-ocr-v2')      # new checkpoint dir
FINAL_MODEL = os.path.join(BASE_DIR, 'florence2-nssf-ocr-final-v2') # new final model
```

Change 5 — Update W&B Run Name

```
WANDB_RUN_NAME = 'florence2-large-v2-retrain'   # distinguish from original run
```

6.4 Complete Re-Fine-Tuning Configuration Example

Here is the complete Cell 2 configuration for a typical re-fine-tuning run on 2,000 new documents:

```
# ── Paths ──────────────────────────────────────────────────────────────────────────────────────
DATA_JSON      = os.path.join(BASE_DIR, 'ultimate_training_dataset_v2.json') # extended
IMAGE_BASE_DIR = BASE_DIR
OUTPUT_DIR     = os.path.join(BASE_DIR, 'florence2-nssf-ocr-v2')
FINAL_MODEL    = os.path.join(BASE_DIR, 'florence2-nssf-ocr-final-v2')

# ── Start from the saved model ──────────────────────────────────────────────────────────────
FLORENCE_BASE = '/workspace/florence2-nssf-ocr-final'

# ── Reduced hyperparameters for re-fine-tuning ──────────────────────────────────────────────
BATCH_SIZE    = 4
GRAD_ACCUM    = 2
NUM_EPOCHS    = 3      # fewer epochs: model already knows the task
LR            = 3e-5   # 3x lower: preserve learned representations
WARMUP_STEPS  = 50

# ── Keep LoRA settings the same ─────────────────────────────────────────────────────────────
LORA_R        = 16
LORA_ALPHA    = 32
LORA_DROPOUT  = 0.05

# ── W&B ────────────────────────────────────────────────────────────────────────────────────────
WANDB_RUN_NAME = 'florence2-nssf-v2-retrain'
```

6.5 Adding New Document Types (New Task Token)

If you want the model to handle a fundamentally different extraction task (not just more NSSF documents), you can register a second task token alongside the original:

```
# In Cell 6, add additional task tokens:
NEW_TASK_TOKEN = '<Extract_NSSF_Table>'   # specialized for table-heavy docs

# Register BOTH tokens
for tok in [TASK_TOKEN, NEW_TASK_TOKEN]:
    if tok not in processor.tokenizer.additional_special_tokens:
        processor.tokenizer.add_special_tokens(
            {'additional_special_tokens': [tok]}
        )
        print(f'Registered {tok}')

# Resize once after adding all tokens
model.resize_token_embeddings(len(processor.tokenizer))
```

> **i Info:** Each task token tells the model which extraction schema to use. Documents sent with <Extract_NSSF_Data> follow the standard schema; documents sent with <Extract_NSSF_Table> could use a table-optimized schema. Keep your dataset entries consistent in which token maps to which output format.

6.6 Re-Fine-Tuning Checklist

12. New images copied to IMAGE_BASE_DIR
13. New entries appended to dataset JSON and validated (Cell 3)
14. FLORENCE_BASE points to saved model path
15. LR reduced to 3e-5
16. OUTPUT_DIR and FINAL_MODEL point to new directories
17. W&B run name updated to distinguish from original run
18. NUM_EPOCHS reduced (3 is usually sufficient for incremental data)

7. Evaluation & Metrics

Cell 11 runs evaluation on the 5% validation split using three complementary metrics. Cell 12 provides failure analysis by document type.

Evaluation Metrics

Metric	Excellent	Acceptable	Needs Work	What It Measures
Exact Match (EM)	> 0.80	0.60 – 0.80	< 0.60	Field-level exact string equality (case/whitespace normalized)
CER	< 0.05	0.05 – 0.15	> 0.15	Character Error Rate — edit distance relative to ground truth length
Fuzzy Match	> 0.90	0.75 – 0.90	< 0.75	fuzz.ratio score — tolerates minor spacing and punctuation differences

4-Stage JSON Recovery Parser

The evaluation uses a robust 4-stage JSON parser that handles model output imperfections:

- **Stage 1:** Standard json.loads() with key whitespace cleanup
- **Stage 2:** Fix trailing commas, bare string entries, and control characters
- **Stage 3:** Truncation recovery — closes JSON at the last complete key-value pair

- **Stage 4:** Regex salvage — extracts flat string pairs even when table arrays are malformed; returns partial dict flagged with _table_parse_failed=True

Troubleshooting Low Scores

Problem	Solution
CER high on handwriting	Increase NUM_EPOCHS to 7–10; add stronger augmentation
Table field scores low	Ensure table ground truth format is consistent; consider dedicated <Extract_NSSF_Table> token
OOM errors during eval	Reduce BENCH_BS (benchmark batch size) in Cell 13
Exact Match low but Fuzzy high	Ground truth has inconsistent field name capitalization — normalize labels
_table_parse_failed in output	Table JSON is malformed — the Stage 4 parser recovered flat fields but table array failed

8. Production Inference

After training completes, use the saved model in your extraction service. Cell 14 provides a ready-to-use snippet.

Basic Inference

```
from transformers import AutoProcessor, AutoModelForCausalLM
from PIL import Image
import torch, json, re

MODEL_PATH = '/workspace/florence2-nssf-ocr-final'
TASK_TOKEN = '<Extract_NSSF_Data>'

processor = AutoProcessor.from_pretrained(MODEL_PATH, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_PATH, trust_remote_code=True, torch_dtype=torch.float16
).to('cuda').eval()

def extract_fields(image_path: str) -> dict:
    image  = Image.open(image_path).convert('RGB')
    inputs = processor(
        text=TASK_TOKEN, images=image, return_tensors='pt'
    ).to('cuda')
    with torch.no_grad():
        out = model.generate(**inputs, max_new_tokens=512, num_beams=3)
    raw = processor.batch_decode(out, skip_special_tokens=True)[0]
    raw = re.sub(r'<[^>]+>', '', raw).strip()
    try:
        return json.loads(raw)
    except json.JSONDecodeError:
```

```
    return {'_raw': raw}   # fallback if JSON parse fails
```

Batched Inference (High Throughput)

For production workloads targeting ≥ 1,200 documents/hour, use the batched inference from Cell 13:

```
def batch_extract(image_paths, batch_size=8, num_beams=3):
    preds = []
    for i in range(0, len(image_paths), batch_size):
        batch = image_paths[i:i+batch_size]
        images  = [Image.open(p).convert('RGB') for p in batch]
        prompts = [TASK_TOKEN] * len(images)
        inp = processor(text=prompts, images=images,
                        return_tensors='pt', padding=True).to('cuda')
        inp['pixel_values'] = inp['pixel_values'].to(torch.float16)
        with torch.no_grad():
            out = model.generate(input_ids=inp['input_ids'],
                pixel_values=inp['pixel_values'],
                max_new_tokens=512, num_beams=num_beams)
        for raw in processor.batch_decode(out, skip_special_tokens=True):
            raw = re.sub(r'<[^>]+>', '', raw).strip()
            try:    preds.append(json.loads(raw))
            except: preds.append({})
    return preds
```

Throughput Optimization Tips

- **Batch size 16:** Biggest single win if VRAM allows (≥ 30 GB)
- **num_beams=3:** Do not increase — higher beams hurt throughput significantly
- **torch.compile():** ~15% speedup on PyTorch 2.0+ (adds ~90s warm-up)
- **4-bit quantization:** ~2x memory reduction via bitsandbytes, ~5% accuracy drop
- **Target throughput:** ≥ 1,200 documents/hour on RTX 4090 with batch_size=8

9. Troubleshooting

Error / Symptom	Cause	Fix
CUDA error: no kernel image	Old PyTorch doesn't support Blackwell sm_120	Re-run Cell 0 to install PyTorch ≥ 2.7.0 from CUDA 12.8 wheels
OverflowError in Rust tokenizer	max_length passed to processor() subtracts image tokens → negative	Do NOT pass max_length/truncation to processor(). Only pass to the label tokenizer.
CUDA out of memory	Batch size too large for VRAM	Halve BATCH_SIZE, double GRAD_ACCUM to keep effective batch the same

Error / Symptom	Cause	Fix
ValueError: hf_transfer not installed	HF_HUB_ENABLE_HF_TRANSFER env var set but package missing	Cell 0 disables it: os.environ['HF_HUB_ENABLE_HF_TRANSFER'] = '0'
Vocab mismatch 51290 vs 51289	Model saved as adapter-only (not merged)	Use merge_and_unload() before save_pretrained() (Cell 10)
Input type / bias type mismatch	Mixed fp16/fp32 dtypes after LoRA merge	Call model.half() after merge_and_unload() in eval loading
AssertionError: model_type davit	Config.json missing vision_config.model_type	Cell 11 auto-patches config.json before loading — ensure this cell is run
gdown download fails	Google Drive quota or auth	Download manually and place in BASE_DIR, then skip Cell 0b
Training loss not decreasing	LR too high or too low	Try LR=5e-5 if loss is unstable; try LR=2e-4 if loss barely moves

10. Quick Reference

Cell Execution Order

Cell	Name	Run Once?	Notes
0	Install Dependencies	Once per environment	RESTART KERNEL after this
0b	Download & Extract Data	Once	Skip if data already on disk
1	Imports & Device	Every session	
2	Configuration	Every session	Edit params here before training
3	Dataset Validation	Before training	Always run; safe to repeat
4	NSSFDocumentDataset	Every session	
5	Data Collator	Every session	
6	Load Model + LoRA	Every session	
7	Split & Build Datasets	Every session	
8	TrainingArguments	Every session	
9	Train	Training run	Set RESUME=True to continue after crash
10	Save Final Model	After training	

Cell	Name	Run Once?	Notes
11	Evaluation	After training	
12	Error Analysis	Optional	
13	Throughput Benchmark	Optional	
14	Production Snippet	Reference	Copy into your service

Key File Paths (RunPod)

```
/workspace/
  ultimate_training_dataset.json   ← training data
  nssf_dataset/                    ← training images
  florence2-nssf-ocr/              ← checkpoints (during training)
  florence2-nssf-ocr-final/        ← final merged model (after training)
  training_loss.png                ← local loss curve backup
```